

- Chapter 5 — Software Application
 - 5.1 Introduction
 - 5.2 Development Environment and Technology Stack
 - 5.2.1 Environment Summary
 - 5.2.2 Justification of Technology Choices
 - 5.3 High-Level System Architecture
 - 5.4 Backend Implementation
 - 5.4.1 Data Fusion: Geo-Spatial Lookup
 - 5.4.2 Real-Time Weather Integration
 - 5.4.3 Demand Engine V4: Passenger Demand Proxy
 - 5.4.4 Union Capacity Assessment
 - 5.4.5 Smart Hustle Algorithm: Zone Classification
 - 5.4.6 API Endpoint and Response Structure
 - 5.5 Frontend User Interface
 - 5.5.1 Application Entry Point and Initialisation
 - 5.5.2 Theme System and Visual Design
 - 5.5.3 Navigation Architecture
 - 5.5.4 Map Screen: Core Interface
 - 5.5.5 Risk Dialog: Zone Classification Display
 - 5.5.6 Navigation Drawer
 - 5.6 Operational Scenarios and Walkthrough
 - Scenario A — RED Zone: Town Hall Union Stand
 - Scenario B — YELLOW Zone: Viharamahadevi Park During Heavy Rain
 - Scenario C — GREEN (Sweet Spot): BMICH During Midday
 - Scenario D — GREEN (Quiet/Freelance Zone): Off-Grid Area at Night
 - 5.7 Real-Time Driver Tracking and State Management
 - 5.7.1 Firebase Driver Location System
 - 5.7.2 Riverpod State Architecture
 - 5.7.3 Lifecycle and Resource Management
 - 5.8 Performance Optimisation
 - 5.9 Security Implementation
 - 5.9.1 Authentication Flow
 - 5.9.2 Route Protection
 - 5.9.3 Data Security Measures
 - 5.10 Summary

Chapter 5 — Software Application

5.1 Introduction

This chapter presents the complete software implementation of **LankaRide**, a real-time risk-assessment mobile application designed for three-wheeler (tuk-tuk) drivers operating in Colombo 7, Sri Lanka. The application materialises the research objectives defined in Chapter 1 and operationalises the methodology established in Chapter 3, transforming the XGBoost traffic prediction model — whose evaluation results were reported in Chapter 4 ($R^2 = 0.99$, RMSE = 2.73 seconds) — into a practical decision-support tool that delivers actionable zone classifications (RED, YELLOW, GREEN) to drivers in the field.

The core engineering challenge addressed in this chapter is the fusion of heterogeneous, real-time data streams — GPS coordinates, live weather telemetry, crowdsourced driver density from Firebase, surveyed union-stand capacities, and XGBoost model predictions — into a single, coherent zone classification that a driver can understand and act upon within seconds. The system architecture follows a client-server paradigm in which a Flutter mobile client communicates with a Flask REST API backend, which in turn orchestrates data fusion from multiple external services before returning a tri-colour risk assessment.

The remainder of this chapter is organised as follows: Section 5.2 summarises the development environment and technology stack; Section 5.3 describes the high-level system architecture; Section 5.4 details the backend implementation including data fusion, demand estimation, and the Smart Hustle decision algorithm; Section 5.5 presents the frontend user-interface design and interaction patterns; Section 5.6 walks through four representative operational scenarios with real data values; Section 5.7 covers real-time driver tracking and state management; Section 5.8 discusses performance optimisation strategies; Section 5.9 addresses security considerations; and Section 5.10 provides a chapter summary.

5.2 Development Environment and Technology Stack

5.2.1 Environment Summary

The following table summarises the principal technologies, frameworks, and services employed in the LankaRide system, together with their roles and version numbers.

Layer	Technology	Version	Role
Mobile Framework	Flutter	3.38.9	Cross-platform UI toolkit
Programming Language (Client)	Dart	3.10.8	Strongly-typed client language
Programming Language (Server)	Python	3.11	Backend and ML pipeline
Web Framework	Flask	3.1.1	Lightweight REST API server
ML Library	XGBoost	2.1.4	Gradient-boosted traffic model
Authentication	Firebase Auth	—	Email/password + guest login
Real-Time Database	Cloud Firestore	—	Driver location streaming
Maps SDK	Google Maps for Flutter	2.12.1	Interactive map rendering
State Management	Riverpod	2.6.1	Reactive state architecture
Navigation	GoRouter	14.8.1	Declarative URL-based routing
HTTP Client	Dio	5.8.0+1	Typed HTTP with interceptors
Weather API	OpenWeatherMap	v2.5	Real-time rainfall telemetry
Geolocation	Geolocator	13.0.2	GPS position stream + permissions
Animation	flutter_animate	4.5.2	Declarative micro-animations

Layer	Technology	Version	Role
UI Effects	glassmorphism_ui	0.3.0	Frosted-glass panel effects
Typography	Google Fonts	6.2.1	Inter typeface family

5.2.2 Justification of Technology Choices

Flutter was selected over native Android/iOS development for its single-codebase advantage, sub-16ms frame rendering via the Skia engine, and hot-reload capability that accelerated iterative UI development. Flutter's widget composition model proved particularly suited to the complex overlay architecture required for the map screen (Google Map layer → driver markers → zone circle → glassmorphic scanner panel → animated risk dialog).

XGBoost was chosen for traffic prediction after systematic comparison with Random Forest and Linear Regression models during the model selection phase (Chapter 4). Its gradient-boosting ensemble approach yielded the highest predictive accuracy ($R^2 = 0.99$) while maintaining inference latency under 5 ms per prediction on the server hardware.

Firebase was selected for its real-time synchronisation capabilities. Firestore's snapshot listener model enables sub-second driver location propagation without requiring the development team to implement WebSocket infrastructure, message queuing, or connection management logic.

Riverpod was preferred over Flutter's built-in Provider or BLoC pattern for its compile-time safety, ref-based dependency injection, and support for family providers — a feature critical for parameterised queries such as `activeDriversProvider.family<List<DriverModel>, Map<String, dynamic>>`.

5.3 High-Level System Architecture

Figure 5.1 illustrates the end-to-end data flow from the driver's GPS reading to the final zone classification displayed on screen. The architecture comprises three tiers: the Flutter

mobile client, the Flask REST API, and external data services (OpenWeatherMap, Firebase, and the trained XGBoost model).

```
Parse error on line 1:  
flowchart TB      sub  
^  
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

Figure 5.1 — High-level system architecture showing the three-tier data flow from GPS acquisition through backend data fusion to risk classification display.

The request-response lifecycle proceeds as follows:

1. The driver taps **"ANALYZE RISK"** on the scanner panel.
2. The Flutter client sends a POST request to `/predict` containing the driver's current GPS coordinates (`latitude`, `longitude`), a client-side rain hint (`rain_level`), and the live count of nearby drivers from Firebase (`union_density`).
3. The Flask server executes four concurrent operations: geo-lookup against the union-stand database, weather fetch from OpenWeatherMap, demand proxy calculation via the XGBoost model, and union capacity evaluation.
4. The Smart Hustle Algorithm synthesises all inputs into a tri-colour zone classification.
5. The response (zone, message, risk score, metadata dictionary) is returned to the client.
6. The Flutter client renders an animated risk dialog with zone-specific colouring, contextual statistics, and actionable advice.

5.4 Backend Implementation

The backend service is implemented as a single Flask application (`server.py`, 282 lines) that orchestrates data fusion from multiple sources. This section examines each component in detail.

5.4.1 Data Fusion: Geo-Spatial Lookup

The geo-spatial lookup function identifies the nearest surveyed union stand to the driver's GPS position using the **Haversine formula** with a 400-metre matching threshold. The

union-stand database (`routes_db.csv`) contains 13 manually surveyed locations in Colombo 7 with their GPS coordinates, observed capacities, and congestion indices.

Table 5.1 — Sample entries from the union-stand database (`routes_db.csv`)

Stand Name	Latitude	Longitude	Capacity	Congestion Index
Independence Square	6.9020	79.8690	15	51
Town Hall	6.9167	79.8640	20	76
General Hospital	6.9180	79.8650	25	91
BMICH	6.9010	79.8700	2	54
Viharamahadevi Park	6.9110	79.8630	7	76
Eye Hospital	6.9190	79.8620	18	91

The Haversine implementation in `perform_geo_lookup()` computes great-circle distance between two latitude-longitude pairs:

```
def perform_geo_lookup(lat, lng):
    """Find nearest union stand within 400m using Haversine formula."""
    nearest = None
    min_dist = float('inf')

    for _, row in routes_df.iterrows():
        # Haversine distance calculation
        R = 6371000 # Earth radius in metres
        phi1, phi2 = math.radians(lat), math.radians(row['latitude'])
        d_phi = math.radians(row['latitude'] - lat)
        d_lambda = math.radians(row['longitude'] - lng)

        a = (math.sin(d_phi / 2) ** 2 +
             math.cos(phi1) * math.cos(phi2) *
             math.sin(d_lambda / 2) ** 2)
        distance = R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

        if distance < min_dist:
            min_dist = distance
            nearest = row

    if min_dist <= 400:
        return nearest, min_dist
    return None, min_dist
```

When no union stand exists within 400 metres, the function returns `None`, and the Smart Hustle Algorithm defaults to a GREEN classification with a "freelance zone" advisory —

directing the driver to operate independently without union stand constraints.

5.4.2 Real-Time Weather Integration

The `get_real_weather()` function queries the OpenWeatherMap API v2.5 with coordinate-based lookup to obtain current rainfall data:

```
def get_real_weather(lat, lng):
    """Fetch real-time weather data from OpenWeatherMap API."""
    api_key = os.getenv('OPENWEATHER_API_KEY', '')
    if not api_key:
        return {'rain_mm': 0.0, 'description': 'unknown'}

    url = f"http://api.openweathermap.org/data/2.5/weather"
    params = {'lat': lat, 'lon': lng, 'appid': api_key, 'units': 'metric'}

    try:
        resp = requests.get(url, params=params, timeout=5)
        data = resp.json()
        rain_mm = data.get('rain', {}).get('1h', 0.0)
        description = data.get('weather', [{}])[0].get('description',
'clear')
        return {'rain_mm': rain_mm, 'description': description}
    except Exception:
        return {'rain_mm': 0.0, 'description': 'unavailable'}
```

Rainfall data serves a dual purpose in the system: (1) as a feature input to the XGBoost model via `rain_1h`, and (2) as an independent trigger in the Smart Hustle Algorithm, where `rain_mm > 2.0` forces a YELLOW classification regardless of other factors, reflecting the empirical observation that moderate rain in Colombo significantly alters traffic patterns and passenger demand.

5.4.3 Demand Engine V4: Passenger Demand Proxy

Since direct passenger demand data is unavailable through any public API, the system employs a novel **Demand Engine V4** that transforms the XGBoost model's traffic intensity prediction into a passenger demand proxy. The approach is grounded in the transport economics principle that traffic congestion correlates with commercial activity and, consequently, with passenger demand for three-wheeler services.

```

def calculate_passenger_demand(traffic_intensity, hour, is_weekend,
rain_mm):
    """
    Demand Engine V4 – Convert traffic intensity into demand proxy.
    Uses time-aware multipliers and event/ghost-town detection.
    """
    # Base demand from traffic intensity
    base_demand = traffic_intensity * 45

    # Time-based multipliers
    if 7 <= hour <= 9:      multiplier = 1.8    # Morning rush
    elif 16 <= hour <= 19:  multiplier = 2.0    # Evening rush (peak)
    elif 10 <= hour <= 15:  multiplier = 1.3    # Midday moderate
    elif 20 <= hour <= 22:  multiplier = 1.1    # Evening wind-down
    else:                   multiplier = 0.6    # Late night / early morning

    # Weekend adjustment
    if is_weekend:
        if 10 <= hour <= 20: multiplier *= 1.2  # Weekend leisure peak
        else:                 multiplier *= 0.8  # Weekend off-peak

    # Rain boost – more people seek rides in rain
    if rain_mm > 0:
        multiplier *= (1.0 + min(rain_mm * 0.1, 0.5)) # Up to 50% boost

    demand = base_demand * multiplier

    # Event detection (traffic_intensity > 2.0 = unusual congestion)
    if traffic_intensity > 2.0:
        demand *= 1.5 # Likely event nearby – surge

    # Ghost-town detection (traffic_intensity < 1.2 = dead zone)
    if traffic_intensity < 1.2:
        demand *= 0.5 # Very low activity – reduce demand

    return round(demand, 1)

```

Table 5.2 — Demand Engine V4 time-based multiplier schedule

Time Window	Hours	Multiplier	Rationale
Morning Rush	07:00 – 09:00	1.8×	Commuter demand peak
Midday	10:00 – 15:00	1.3×	Moderate commercial activity
Evening Rush	16:00 – 19:00	2.0×	Highest demand period
Evening Wind-down	20:00 – 22:00	1.1×	Declining but present
Late Night	23:00 – 06:00	0.6×	Minimal demand

The Demand Engine also incorporates two anomaly detection mechanisms:

- **Event Detection:** When `traffic_intensity > 2.0`, the demand is boosted by 50%, reflecting scenarios such as concerts, cricket matches at SSC grounds, or government functions at BMICH that generate abnormal passenger volumes.
- **Ghost-Town Detection:** When `traffic_intensity < 1.2`, demand is halved, preventing drivers from being directed to genuinely inactive areas.

5.4.4 Union Capacity Assessment

The `get_active_union_capacity()` function evaluates the saturation level of the nearest union stand by comparing observed driver density against surveyed capacity:

```
def get_active_union_capacity(capacity, union_density):
    """Assess union stand saturation. Returns capacity and overflow flag."""
    if capacity <= 0:
        return 0, False
    overflow = union_density >= capacity
    return capacity, overflow
```

A critical threshold is applied: if `capacity >= 15` (indicating a large, well-established stand), the system immediately classifies the zone as RED. This rule reflects field observations during the requirements-gathering phase (Chapter 3), where large stands were consistently found to be controlled by established driver syndicates, making them hostile environments for freelance or new drivers.

5.4.5 Smart Hustle Algorithm: Zone Classification

The **Smart Hustle Algorithm** is the system's central decision function, implementing a four-tier priority cascade that synthesises all data inputs into a final zone classification. The algorithm is named to reflect its purpose: helping drivers make smart decisions about where to seek passengers (hustle) safely.

```
Parse error on line 1:
flowchart TD      Sta
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

Figure 5.2 — Smart Hustle Algorithm decision flowchart showing the four-tier priority cascade.

The Python implementation of the `decide_smart_zone()` function:

```
def decide_smart_zone(capacity, rain_mm, traffic_intensity, demand,
                      nearby_drivers, hour, poi_density):
    """
    Smart Hustle Algorithm – 4-tier decision cascade.
    Returns: (zone, message, risk_score)
    """
    risk_score = 0.5 # Neutral baseline

    # Night-time capacity reduction (22:00 – 06:00)
    if hour >= 22 or hour < 6:
        capacity = int(capacity * 0.8) # 20% reduction

    # Firebase driver count boost
    if nearby_drivers > 5:
        capacity = min(capacity + nearby_drivers, 30)

    # === TIER 1: Union Risk (highest priority) ===
    if capacity >= 15:
        risk_score = 0.85 + min(capacity / 100, 0.14)
        return ("RED",
                f"⚠ Major union stand ({capacity} drivers). "
                f"High syndicate control – AVOID this area.",
                risk_score)

    # === TIER 2: Gridlock / Weather ===
    if rain_mm > 2.0 or demand > 100:
        risk_score = 0.55 + min(rain_mm * 0.05, 0.2)
        reason = "Heavy rain" if rain_mm > 2.0 else "High demand overflow"
        return ("YELLOW",
                f"☁ {reason} detected. "
                f"OPPORTUNITY – passengers seeking rides, but drive
safely.",
                risk_score)

    # === TIER 3: Active Crowd ===
    if traffic_intensity > 1.5 and demand > 60:
        risk_score = 0.45 + min(demand / 500, 0.15)
        return ("YELLOW",
                f"👥 Busy area (demand: {demand:.0f}). "
                f"Moderate competition – maintain 500m buffer from stand.",
                risk_score)

    # === TIER 4: Demand Score ===
    if demand > 40:
        risk_score = 0.25 + min(demand / 400, 0.15)
        return ("GREEN",
                f"✅ Sweet spot! Good demand ({demand:.0f}) with low risk. "
                f"Ideal location for pickups.",
                risk_score)
```

```
# Default: Quiet Zone
risk_score = 0.1 + min(traffic_intensity * 0.05, 0.1)
return ("GREEN",
        f"🌿 Quiet zone (demand: {demand:.0f}). "
        f"Safe area but limited passengers. Consider relocating.",
        risk_score)
```

The four tiers are evaluated in strict priority order:

1. **Tier 1 — Union Risk** (capacity ≥ 15): Immediate RED classification. Large union stands represent the highest risk to individual drivers due to established syndicate control. The risk score is computed as $0.85 + \text{capacity}/100$, capped at 0.99.
2. **Tier 2 — Gridlock/Weather** (rain > 2mm OR demand > 100): YELLOW classification. Heavy rain or demand overflow creates opportunity (passengers seek rides) but also risk (dangerous driving conditions, increased competition). The 500m buffer advisory is included.
3. **Tier 3 — Active Crowd** (traffic_intensity > 1.5 AND demand > 60): YELLOW classification. The area is commercially active with moderate competition, representing a balanced opportunity.
4. **Tier 4 — Demand Score** (demand > 40): GREEN classification with "sweet spot" advisory. Sufficient passenger demand exists without dangerous competition levels.
5. **Default — Quiet Zone** (demand ≤ 40): GREEN classification with relocation advisory. The area is safe but unprofitable.

5.4.6 API Endpoint and Response Structure

The `/predict` endpoint receives a POST request with the driver's context and returns a comprehensive JSON response:

Request payload:

```
{
  "latitude": 6.9167,
  "longitude": 79.8640,
  "rain_level": 0.0,
  "union_density": 3.0
}
```

Response payload:

```
{
  "zone": "RED",
  "message": "⚠️ Major union stand (20 drivers). High syndicate control – AVOID this area.",
  "risk_score": 0.95,
  "metadata": {
    "location_name": "Town Hall",
    "distance_m": 89.2,
    "capacity": 20,
    "congestion_index": 76,
    "traffic_intensity": 1.68,
    "demand_proxy": 72.5,
    "demand_level": "High demand",
    "rain_mm": 0.0,
    "weather_desc": "clear sky",
    "poi_density": 76,
    "model_prediction_s": 45.2,
    "nearby_drivers": 3
  }
}
```

The metadata dictionary provides transparency into the decision-making process, enabling the frontend to display contextual statistics (traffic intensity, rain level, union capacity) that help drivers understand *why* a particular zone classification was assigned. This interpretability was identified as a key requirement during stakeholder interviews (Chapter 3).

5.5 Frontend User Interface

5.5.1 Application Entry Point and Initialisation

The Flutter application initialises through a carefully sequenced startup process defined in `main.dart`:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // 1. Load environment variables (.env)
  await dotenv.load(fileName: ".env");

  // 2. Initialize Firebase
```

```

await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);

// 3. Lock portrait orientation
await SystemChrome.setPreferredOrientations([
  DeviceOrientation.portraitUp,
  DeviceOrientation.portraitDown,
]);

// 4. Launch app with Riverpod state container
runApp(const ProviderScope(child: LankaRideApp()));
}

```

The **ProviderScope** widget at the root of the widget tree establishes the Riverpod dependency injection container, ensuring that all providers (authentication state, map controller, driver data, theme mode) share a single lifecycle scope.

5.5.2 Theme System and Visual Design

LankaRide implements a dual-theme system (light and dark modes) using a custom **AppTheme** class that defines a cohesive colour palette and typography system:

Table 5.3 — Application colour palette

Colour Token	Hex Value	Usage
primary	#2563EB	Buttons, links, active elements
accent	#10B981	GREEN zone, success states
error	#EF4444	RED zone, error states
warning	#F59E0B	YELLOW zone, caution states
background	#FAFAFA	Light mode scaffold
dark	#1A1B23	Dark mode scaffold
surface	#FFFFFF	Card and panel backgrounds
surfaceDark	#262837	Dark mode elevated surfaces

The typography system uses the **Inter** typeface family loaded via **google_fonts**, providing 8 text styles from **displayLarge** (32px, weight 800) down to **labelLarge** (14px, weight 600). The **ThemeModeNotifier** persists the user's theme preference to

`SharedPreferences` and exposes a `toggleTheme()` method for the navigation drawer toggle.

5.5.3 Navigation Architecture

The application employs **GoRouter** for declarative, URL-based navigation with built-in authentication guards:

```
final routerProvider = Provider<GoRouter>((ref) {
  final isAuthenticated = ref.watch(isAuthenticatedProvider);

  return GoRouter(
    initialLocation: '/',
    redirect: (context, state) async {
      final authenticated = isAuthenticated.value ?? false;

      // Protect map and profile routes
      if (!authenticated &&
          (state.matchedLocation == '/map' ||
           state.matchedLocation == '/profile')) {
        return '/login';
      }

      // Redirect authenticated users away from auth screens
      if (authenticated &&
          (state.matchedLocation == '/login' ||
           state.matchedLocation == '/onboarding')) {
        return '/map';
      }
      return null;
    },
    routes: [
      GoRoute(path: '/',          builder: (_, __) => const SplashScreen()),
      GoRoute(path: '/onboarding', builder: (_, __) => const
OnboardingScreen()),
      GoRoute(path: '/login',     builder: (_, __) => const LoginScreen()),
      GoRoute(path: '/signup',    builder: (_, __) => const SignupScreen()),
      GoRoute(path: '/profile',   builder: (_, __) => const
ProfileScreen()),
      GoRoute(path: '/map',       builder: (_, __) => const MapScreen()),
    ],
  );
});
```

Table 5.4 — Application screen inventory

Route	Screen	Auth Required	Purpose
/	SplashScreen	No	Branding + auto-navigation
/onboarding	OnboardingScreen	No	First-launch feature tour
/login	LoginScreen	No	Email/password authentication
/signup	SignupScreen	No	New account registration
/forgot-password	ForgotPasswordScreen	No	Password reset flow
/map	MapScreen	Yes	Core risk assessment interface
/profile	ProfileScreen	Yes	Driver profile management

5.5.4 Map Screen: Core Interface

The map screen (**MapScreen**, 814 lines) is the application's primary interface, implemented as a **ConsumerStatefulWidget** to access both Riverpod providers and local lifecycle methods. The screen architecture consists of four visual layers stacked via a **Stack** widget:

Layer 1 — Google Map: Renders the interactive map with custom JSON styling (light/dark variants loaded from **assets/map_style_light.json** and **assets/map_style.json**). The map displays the user's blue marker, nearby driver markers (custom tuk-tuk emoji icons rendered via Canvas API), and the zone circle overlay.

Layer 2 — Driver Markers: Active drivers from Firebase are displayed as custom circular markers featuring a green background with a 🚗 emoji, rendered programmatically using Flutter's **Canvas** and **PictureRecorder** APIs:

```

Future<void> _loadTukTukIcon() async {
  final pictureRecorder = ui.PictureRecorder();
  final canvas = Canvas(pictureRecorder);
  const double size = 120.0;

  // Green circular background
  canvas.drawCircle(
    const Offset(size / 2, size / 2), size / 2,
    Paint()..color = const Color(0xFF10B981)..style = PaintingStyle.fill,
  );

  // White border
  canvas.drawCircle(
    const Offset(size / 2, size / 2), size / 2 - 2,
    Paint()..color = Colors.white..style = PaintingStyle.stroke..strokeWidth
= 4.0,
  );

  // Tuk-tuk emoji
  final textPainter = TextPainter(
    text: const TextSpan(text: '🚲', style: TextStyle(fontSize: 60.0)),
    textDirection: TextDirection.ltr,
  )..layout();
  textPainter.paint(canvas,
    Offset((size - textPainter.width) / 2, (size - textPainter.height) /
2));

  final image = await pictureRecorder.endRecording().toImage(size.toInt(),
size.toInt());
  final byteData = await image.toByteData(format: ui.ImageByteFormat.png);
  _tukTukIcon = BitmapDescriptor.fromBytes(byteData!.buffer.asUint8List());
}

```

Each driver marker displays an **InfoWindow** with the driver's license plate, star rating, and trip count:

 ABC-1234
 4.8 • 156 trips

Layer 3 — Zone Circle: A translucent coloured circle overlaid on the map after risk analysis, using the zone colour (red/yellow/green) with 20% opacity fill and a solid 2px stroke.

Layer 4 — Scanner Panel: A glassmorphic floating panel positioned at the bottom of the screen, featuring a frosted-glass effect via **GlassContainer** with 25px blur and 25% opacity. The panel contains:

- A rotating radar icon (**Icons.radar**) with continuous 3-second rotation animation
- The title "Zone Scanner" with the heading style
- A descriptive subtitle: "Analyze your current location for risk assessment"
- An "ANALYZE RISK" elevated button with continuous shimmer animation

 **Screenshot Placeholder** — **Figure 5.3:** *Capture the LankaRide map screen showing the Google Map with custom dark styling, at least 2-3 tuk-tuk driver markers (green circles with 🚲 emoji), the user's blue location marker, and the glassmorphic Scanner Panel at the bottom with the "ANALYZE RISK" button. Ensure the rotating radar icon is visible.*

5.5.5 Risk Dialog: Zone Classification Display

The **RiskDialog** widget (323 lines) presents the zone classification result as a full-screen animated dialog with zone-specific theming. The dialog renders sequentially using staggered **flutter_animate** animations:

Table 5.5 — Risk Dialog animation sequence

Sequence	Delay	Duration	Element	Effect
1	0 ms	400 ms	Zone icon	FadeIn + Pulse + Shimmer
2	200 ms	400 ms	Subtitle	FadeIn
3	300 ms	400 ms	Message container	FadeIn
4	350 ms	400 ms	Location badge	FadeIn
5	400 ms	400 ms	Stats row (Traffic / Rain / Union)	FadeIn
6	450 ms	400 ms	Demand level bar	FadeIn
7	500 ms	400 ms	Risk score percentage	FadeIn

Sequence	Delay	Duration	Element	Effect
8	550 ms	400 ms	Buffer advisory (YELLOW only)	FadeIn
9	600 ms	400 ms	"GOT IT" button	FadeIn + SlideY

The zone-to-visual mapping is implemented through a conditional system:

```
// Zone colour mapping
final zoneColor = response.isRedZone
    ? AppTheme.error // #EF4444
    : response.isYellowZone
        ? AppTheme.warning // #F59E0B
        : AppTheme.accent; // #10B981

// Zone icon mapping
final zoneIcon = response.isRedZone
    ? Icons.dangerous
    : response.isYellowZone
        ? Icons.warning_amber_rounded
        : Icons.check_circle;

// Zone title mapping
final title = response.isRedZone
    ? 'DANGER!'
    : response.isYellowZone
        ? 'OPPORTUNITY!'
        : 'SAFE ZONE';
```

The dialog also displays three statistical chips showing real-time values: **Traffic** (e.g., "1.7x"), **Rain** (e.g., "2.5mm"), and **Union** capacity (e.g., "20"). For YELLOW zones, an additional advisory banner recommends maintaining a 500-metre buffer from the nearest stand.



Screenshot Placeholder — Figure 5.4: Capture three side-by-side screenshots of the Risk Dialog showing each zone type: (a) RED zone with "DANGER!" title and red colouring, (b) YELLOW zone with "OPPORTUNITY!" title, amber colouring, and the 500m buffer advisory, (c) GREEN zone with "SAFE ZONE" title and emerald colouring. Ensure the stats row (Traffic / Rain / Union) is visible in each.

5.5.6 Navigation Drawer

The navigation drawer provides access to secondary functions: profile management, theme toggle (light/dark mode switch), FAQ information, and sign-out functionality. The drawer header displays the user's profile photo (retrieved from the local file system or a default avatar), display name, and email address from Firebase Auth.

 **Screenshot Placeholder** — **Figure 5.5:** *Capture the navigation drawer open, showing the user profile header, menu items (Profile, Toggle Theme, FAQ, Sign Out), and the dark theme toggle in action.*

5.6 Operational Scenarios and Walkthrough

This section demonstrates the system's behaviour across four representative scenarios using real data values from the union-stand database and actual algorithm thresholds. These scenarios collectively validate that the system correctly handles the full range of operational conditions a tuk-tuk driver may encounter in Colombo 7.

Scenario A — RED Zone: Town Hall Union Stand

Context: A new freelance driver approaches the Town Hall area at 14:00 on a weekday. The Town Hall stand is one of the largest in Colombo 7 with 20 surveyed tuk-tuks.

Input Parameters:

Parameter	Value	Source
Latitude	6.9167	Driver GPS
Longitude	79.8640	Driver GPS
Hour	14	System clock
Rain (mm)	0.0	OpenWeatherMap
Nearby drivers (Firebase)	3	Firestore query

Algorithm Trace:

- 1. **Geo-Lookup:** Haversine distance to Town Hall stand = 89m (within 400m threshold)
→ Match found. Capacity = 20, Congestion Index = 76.
- 2. **Weather:** Clear sky, rain_mm = 0.0.
- 3. **Demand Engine V4:** traffic_intensity = 1.68, base_demand = 1.68 × 45 = 75.6.
Hour 14 → midday multiplier 1.3×. Final demand = 75.6 × 1.3 = 98.3.
- 4. **Smart Hustle Tier 1:** capacity (20) ≥ 15 → **RED classification triggered immediately.**
- 5. **Risk Score:** 0.85 + (20/100) = 0.85 + 0.20 = **0.95** (capped at 0.99).

Output:

- **Zone:** RED
- **Message:** "⚠ Major union stand (20 drivers). High syndicate control — AVOID this area."
- **Risk Score:** 95%
- **Action:** Driver receives RED dialog with danger icon. Advised to avoid the area entirely.

📸 **Screenshot Placeholder — Figure 5.6:** Capture the map screen showing a RED zone circle around the Town Hall area, followed by the RED Risk Dialog displaying "DANGER!", risk score 95%, and the statistics row.

Scenario B — YELLOW Zone: Viharamahadevi Park During Heavy Rain

Context: A driver is near Viharamahadevi Park at 17:30 during monsoon rain. The park stand has a moderate capacity of 7 tuk-tuks.

Input Parameters:

Parameter	Value	Source
Latitude	6.9110	Driver GPS
Longitude	79.8630	Driver GPS
Hour	17	System clock
Rain (mm)	4.5	OpenWeatherMap
Nearby drivers (Firestore)	2	Firestore query

Algorithm Trace:

- 1. **Geo-Lookup:** Viharamahadevi Park stand matched at 125m. Capacity = 7, Congestion Index = 76.
- 2. **Smart Hustle Tier 1:** capacity (7) < 15 → Pass.
- 3. **Demand Engine V4:** traffic_intensity = 1.52. base_demand = 1.52 × 45 = 68.4. Hour 17 → evening rush multiplier 2.0×. Rain boost: 1.0 + min(4.5 × 0.1, 0.5) = 1.45. Final demand = 68.4 × 2.0 × 1.45 = **198.4**.
- 4. **Smart Hustle Tier 2:** rain_mm (4.5) > 2.0 → **YELLOW classification triggered**.
- 5. **Risk Score:** 0.55 + min(4.5 × 0.05, 0.2) = 0.55 + 0.2 = **0.75**.

Output:

- **Zone:** YELLOW
- **Message:** "🌧️ Heavy rain detected. OPPORTUNITY — passengers seeking rides, but drive safely."
- **Risk Score:** 75%
- **Action:** Driver sees YELLOW dialog with "OPPORTUNITY!" title. The 500m buffer advisory appears. The high demand (198.4) suggests many passengers are seeking rides in the rain.

📸 **Screenshot Placeholder — Figure 5.7:** Capture the YELLOW Risk Dialog showing "OPPORTUNITY!" title, rain statistics (4.5mm), and the 500m buffer advisory banner at the bottom.

Scenario C — GREEN (Sweet Spot): BMICH During Midday

Context: A driver is near BMICH at 11:00 on a weekday. BMICH has only 2 surveyed tuk-tuks — one of the smallest stands in the dataset.

Input Parameters:

Parameter	Value	Source
Latitude	6.9010	Driver GPS
Longitude	79.8700	Driver GPS
Hour	11	System clock

Parameter	Value	Source
Rain (mm)	0.0	OpenWeatherMap
Nearby drivers (Firebase)	1	Firestore query

Algorithm Trace:

- 1. **Geo-Lookup:** BMICH stand matched at 95m. Capacity = 2, Congestion Index = 54.
- 2. **Smart Hustle Tier 1:** capacity (2) < 15 → Pass.
- 3. **Demand Engine V4:** traffic_intensity = 1.35. base_demand = 1.35 × 45 = 60.75.
Hour 11 → midday multiplier 1.3×. Final demand = 60.75 × 1.3 = **79.0**.
- 4. **Smart Hustle Tier 2:** rain (0.0) ≤ 2.0 AND demand (79.0) ≤ 100 → Pass.
- 5. **Smart Hustle Tier 3:** traffic_intensity (1.35) ≤ 1.5 → Pass (first condition not met).
- 6. **Smart Hustle Tier 4:** demand (79.0) > 40 → **GREEN (Sweet Spot) classification triggered.**
- 7. **Risk Score:** 0.25 + min(79.0/400, 0.15) = 0.25 + 0.15 = **0.40**.

Output:

- **Zone:** GREEN
- **Message:** "✅ Sweet spot! Good demand (79) with low risk. Ideal location for pickups."
- **Risk Score:** 40%
- **Action:** Driver sees GREEN dialog with "SAFE ZONE" title and check-circle icon. Encouraged to operate in this area.



Screenshot Placeholder — *Figure 5.8: Capture the GREEN Risk Dialog showing "SAFE ZONE" title, the demand level indicator, and the low risk score percentage.*

Scenario D — GREEN (Quiet/Freelance Zone): Off-Grid Area at Night

Context: A driver is in a residential area with no nearby union stands at 23:00 (late night).

Input Parameters:

Parameter	Value	Source
Latitude	6.9050	Driver GPS
Longitude	79.8580	Driver GPS
Hour	23	System clock
Rain (mm)	0.0	OpenWeatherMap
Nearby drivers (Firestore)	0	Firestore query

Algorithm Trace:

- 1. **Geo-Lookup:** No union stand within 400m → **Freelance Zone** (GREEN returned immediately).
- 2. No further tiers evaluated — the system bypasses the full decision cascade.

Output:

- **Zone:** GREEN
- **Message:** "🌿 You're in a freelance zone! No nearby union stands — operate freely."
- **Risk Score:** 15%
- **Action:** Driver sees GREEN dialog indicating no union presence. Free to seek passengers without competition concerns but should be aware of low late-night demand.

📸 **Screenshot Placeholder** — **Figure 5.9:** Capture the GREEN "Freelance Zone" dialog showing the safe zone indication with the low risk score.

Summary of Scenario Outcomes:

Scenario	Location	Zone	Risk Score	Primary Trigger
A	Town Hall	RED	95%	Tier 1: capacity (20) ≥ 15
B	Viharamahadevi Park	YELLOW	75%	Tier 2: rain (4.5mm) > 2.0
C	BMICH	GREEN ★	40%	Tier 4: demand (79) > 40

Scenario	Location	Zone	Risk Score	Primary Trigger
D	Off-grid residential	GREEN	15%	No stand within 400m

5.7 Real-Time Driver Tracking and State Management

5.7.1 Firebase Driver Location System

LankaRide implements a real-time driver tracking system using Cloud Firestore's snapshot listeners. When a driver opens the map screen, their location is continuously streamed to Firestore and, simultaneously, they receive real-time updates about nearby active drivers.

Table 5.6 — Firestore document schema (**users** collection)

Field	Type	Description
latitude	number	Current GPS latitude
longitude	number	Current GPS longitude
is_live	boolean	Whether driver is currently active
last_updated	timestamp	Server timestamp of last update
name	string	Driver display name
email	string	Registration email
license_plate	string	Vehicle registration number
vehicle_color	string	Tuk-tuk colour (e.g., "Yellow")
rating	number	Average star rating (1.0–5.0)
total_trips	number	Lifetime trip count
photo_path	string	Local path to profile photo
created_at	timestamp	Account creation date

The location tracking pipeline operates with three optimisation mechanisms:

1. **Distance Filter:** The GPS stream uses a 50-metre distance filter (`distanceFilter: 50`), meaning the device only fires a location update when the driver has moved at least 50 metres. This reduces unnecessary processing and battery consumption by approximately 80% compared to continuous polling.
2. **Debounced Firestore Writes:** Raw GPS events are debounced with a 3-second timer before writing to Firestore, preventing write quota exhaustion during rapid movement:

```
// Debounced Firestore update (3-second cooldown)
_debounceTimer?.cancel();
_debounceTimer = Timer(const Duration(seconds: 3), () {
  if (mounted) _updateUserLocation(position);
});
```

3. **Backup Heartbeat:** A 60-second periodic timer ensures the driver's location is updated at least once per minute, even when stationary, keeping the `last_updated` timestamp fresh for the online/offline detection system.

5.7.2 Riverpod State Architecture

The application's state management is built on Riverpod's provider system, organising state into clearly defined, reactive providers:

Table 5.7 — Key Riverpod providers

Provider	Type	Purpose
<code>authStateProvider</code>	StreamProvider	Firebase Auth user stream
<code>isAuthenticatedProvider</code>	FutureProvider	Combined auth check (Firebase + session)
<code>themeModeProvider</code>	StateNotifierProvider	Light/dark theme persistence
<code>routerProvider</code>	Provider	GoRouter with auth redirects
<code>mapControllerProvider</code>	StateProvider	Google Map controller reference

Provider	Type	Purpose
<code>currentLocationProvider</code>	StateProvider	Latest GPS coordinates
<code>driverMarkersProvider</code>	StateProvider	Set of active driver markers
<code>activeDriversNearbyProvider</code>	StateProvider	Nearby driver model list
<code>zoneCircleProvider</code>	StateProvider	Zone classification circle overlay
<code>isAnalyzingProvider</code>	StateProvider	Risk analysis loading state
<code>activeDriversProvider</code>	StreamProvider.family	Firestore driver stream (parameterised)
<code>onlineDriversCountProvider</code>	StreamProvider	Total online driver count
<code>firebaseProvider</code>	Provider	Firestore instance singleton
<code>apiServiceProvider</code>	Provider	Dio-based API client singleton

The `activeDriversProvider` uses Riverpod's `.family` modifier to accept parameterised queries with latitude, longitude, and radius:

```
final activeDriversProvider =
  StreamProvider.family<List<DriverModel>, Map<String, dynamic>>((ref,
params) {
  final db = ref.watch(firebaseProvider);
  final userLat = params['latitude'] as double;
  final userLng = params['longitude'] as double;
  final radiusKm = params['radiusKm'] as double? ?? 5.0;

  return db.collection('users')
    .where('is_live', isEqualTo: true)
    .orderBy('last_updated', descending: true)
    .limit(50) // Performance cap
    .snapshots()
    .map((snapshot) => snapshot.docs
      .map((doc) => DriverModel.fromFirestore(doc))
      .where((driver) {
        // Euclidean approximation: 1° ≈ 111 km
```

```

        final distance = sqrt(
            pow(userLat - driver.latitude, 2) +
            pow(userLng - driver.longitude, 2)) * 111;
        return distance <= radiusKm;
    }).toList());
});

```

This approach enables the system to efficiently query only nearby drivers rather than loading the entire collection, while Firestore's snapshot listener ensures real-time updates without polling.

5.7.3 Lifecycle and Resource Management

The **MapScreen** manages multiple asynchronous resources (GPS stream, Firestore listener, timers, map controller) that must be properly disposed to prevent memory leaks. The **dispose()** method cancels all subscriptions and timers, while the **mounted** guard (checked at 9 critical points throughout the widget) prevents state updates on disposed widgets — a common cause of runtime exceptions in Flutter applications:

```

@override
void dispose() {
    _mapController?.dispose();
    _setUserOffline();           // Mark user as inactive in Firestore
    _locationUpdateTimer?.cancel(); // Cancel 60s heartbeat timer
    _positionStream?.cancel();    // Cancel GPS position stream
    _debounceTimer?.cancel();     // Cancel 3s write debounce timer
    super.dispose();
}

```

When the driver exits the map screen, **_setUserOffline()** sets **is_live** to **false** in their Firestore document, ensuring they are no longer displayed as active on other drivers' maps.

5.8 Performance Optimisation

Several performance optimisations were implemented to ensure smooth operation on mid-range Android devices common among Sri Lankan tuk-tuk drivers.

Table 5.8 — Performance optimisation strategies

Optimisation	Technique	Impact
Driver marker caching	Reuse single <code>BitmapDescriptor</code> instance across all markers	Eliminates per-marker image generation overhead
Marker diff algorithm	Compare existing vs. new driver ID sets before <code>setState()</code>	Prevents unnecessary widget rebuilds
Location distance filter	50m minimum movement threshold	Reduces GPS event frequency by ~80%
Firestore write debounce	3-second <code>Timer</code> before writes	Prevents Firestore write quota saturation
Driver reload throttle	10-second minimum interval between Firebase queries	Limits redundant Firestore document reads
Firestore query limit	<code>.limit(50)</code> on active driver queries	Caps document reads per snapshot event
HTTP timeout	10-second connect + receive timeouts on Dio	Prevents hung requests blocking the UI thread
Map style pre-loading	Load both dark/light JSON styles in <code>initState()</code>	Eliminates style-switch file I/O latency
Canvas icon rendering	Programmatic <code>PictureRecorder</code> marker generation	Avoids asset loading and decoding overhead
Camera movement	<code>moveCamera()</code> instead of <code>animateCamera()</code> for tracking	Reduces GPU compositing and frame jank

The combination of these strategies ensures the application maintains 60fps map scrolling while simultaneously streaming driver locations from Firebase and updating marker positions.

5.9 Security Implementation

5.9.1 Authentication Flow

The application supports two authentication modes:

1. **Firestore Email/Password Authentication:** Full account creation with email verification, password reset via the `ForgotPasswordScreen`, and persistent session tokens managed by the Firebase SDK. User credentials are never stored locally.
2. **Guest Mode:** A lightweight session-based login for demonstration purposes, generating a unique guest identifier (`guest_<timestamp>`) with session validation via `SharedPreferences`. Guest sessions grant map access but do not create Firestore user documents.

The `isAuthenticatedProvider` checks both modes in priority order:

```
final isAuthenticatedProvider = FutureProvider<bool>((ref) async {  
  // Priority 1: Check Firestore auth  
  final authState = await ref.watch(authStateProvider.future);  
  if (authState != null) return true;  
  
  // Priority 2: Check guest session  
  final prefs = await SharedPreferences.getInstance();  
  final userId = prefs.getString(AppConstants.userIdKey);  
  final hasValidSession = prefs.getBool('has_valid_session') ?? false;  
  return userId != null && userId.isNotEmpty && hasValidSession;  
});
```

5.9.2 Route Protection

GoRouter's `redirect` function acts as a middleware-style route guard, preventing unauthenticated access to protected screens. Any attempt to navigate to `/map` or `/profile` without valid authentication redirects the user to `/login`. Conversely, authenticated users attempting to access login or onboarding screens are automatically redirected to `/map`.

5.9.3 Data Security Measures

Security Measure	Implementation	Purpose
API key isolation	<code>.env</code> file excluded via <code>.gitignore</code>	Prevents credential exposure in version control
Request timeouts	10s connect + 10s receive on all HTTP calls	Prevents indefinite request blocking
Graceful degradation	Server-side <code>try/except</code> with safe defaults	External service failures produce defaults, not crashes
Merge writes	<code>SetOptions(merge: true)</code> on all Firestore writes	Prevents accidental document overwrites
Firebase Security Rules	Firestore rules restrict document access to authenticated users	Prevents unauthorised data access
Platform detection	Auto-detection of Android emulator vs. iOS for base URL	Ensures correct API endpoint without hardcoding

5.10 Summary

This chapter has presented the complete software implementation of LankaRide, demonstrating how the research artefacts from preceding chapters — the surveyed union-stand dataset (Chapter 3), the XGBoost traffic prediction model (Chapter 4), and the stakeholder-derived requirements — were synthesised into a functional, real-time mobile application.

The key technical contributions implemented in this chapter include:

1. **Multi-Source Data Fusion:** Integration of GPS coordinates, OpenWeatherMap rainfall data, Firestore driver density, and CSV-based union-stand surveys into a unified prediction pipeline through the Flask backend's `/predict` endpoint.
2. **Demand Engine V4:** A novel proxy calculation that transforms XGBoost traffic intensity predictions into passenger demand estimates using time-aware multipliers ($0.6\times - 2.0\times$), weekend adjustments, rain boost (up to 50%), and anomaly detection (event surge at intensity > 2.0 ; ghost-town detection at intensity < 1.2).

3. **Smart Hustle Algorithm:** A four-tier priority cascade that converts raw data into actionable tri-colour zone classifications, with each tier addressing a distinct risk category (union control → weather/gridlock → crowd activity → demand level).
4. **Real-Time Architecture:** Sub-second driver tracking via Firestore snapshot listeners with three optimisation layers: 50m distance filter, 3-second write debounce, and 60-second heartbeat backup.
5. **Interpretable UI:** A risk dialog that displays not only the zone classification but also the underlying data inputs (traffic intensity, rain level, union capacity, demand level, risk score percentage), promoting driver understanding and trust in the system's recommendations.

The four operational scenarios (Section 5.6) demonstrated that the system correctly differentiates between dangerous union-controlled zones (RED, 95% risk), high-opportunity weather events (YELLOW, 75% risk), optimal operating locations (GREEN sweet spot, 40% risk), and low-activity freelance areas (GREEN quiet zone, 15% risk). Chapter 6 will present the system evaluation, including user acceptance testing results and performance benchmarking.